

Natural Language Processing

Mini Project: Movie Review Sentiment Analyser

Intern Assignment

This project is designed to give you hands-on experience with the full NLP pipeline — from raw text to a working sentiment classifier. You will preprocess text, experiment with different representations, build models, and evaluate your results. You are expected to explain your decisions, not just write code.

Dataset

IMDB Movie Reviews dataset — 50,000 movie reviews labelled as positive or negative.

Load it directly using:

```
from tensorflow.keras.datasets import imdb
```

Or download from:

<https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews>

Part 1: Text Preprocessing *Clean and prepare the raw text for modelling*

- Convert all text to lowercase
- Remove punctuation, special characters, and HTML tags
- Tokenise the text using NLTK or spaCy
- Remove stop words
- Apply lemmatisation
- Print 3 examples showing the original review and the cleaned version side by side

 *What to explain: Why did you make each preprocessing decision? Did you remove stop words? Why or why not?*

Part 2: Exploratory Analysis *Before building any model, understand your data*

- How many reviews are positive vs negative? Is the dataset balanced?
- What is the average review length (in words) before and after preprocessing?
- Plot a word cloud for positive reviews and a separate one for negative reviews
- Plot the top 20 most frequent words after preprocessing
- What do you notice? Are there any surprising words?

 *What to explain: What did you learn about the data from this analysis?*

Part 3: Text Representation *Convert your cleaned text into a format a model can use*

- Represent the text using TF-IDF (use sklearn's TfidfVectorizer)
- Represent the text using Word Embeddings (use a Keras Embedding layer or pre-trained GloVe vectors)
- Print the TF-IDF scores for 5 words in one review and explain what the scores mean

📌 *What to explain: What is the difference between TF-IDF and word embeddings? Which do you think will work better for this task and why?*

Part 4: Build Two Models *Train and compare two different approaches*

Build and train two different models:

- Model 1 — Classical: Logistic Regression with TF-IDF features
- Model 2 — Deep Learning: LSTM with an Embedding layer

For each model:

- Train on the training set
- Evaluate on the test set
- Report accuracy, precision, recall, and F1 score
- Plot the confusion matrix

📌 *What to explain: How did you decide on the architecture for your LSTM? How many layers, units, and dropout did you use?*

Part 5: Compare and Reflect *Analyse your results and draw conclusions*

- Which model performed better overall?
- Which model was faster to train?
- Look at the confusion matrix — which type of error is more common: false positives or false negatives? Does this matter for this task?
- Show 3 examples where your best model got it wrong. Why do you think it failed?

📌 *What to explain: If you had to deploy one of these models in a real product, which would you choose and why?*

Part 6: Bonus (Optional) *For those who want to go further*

- Load a pre-trained BERT model using Hugging Face and fine-tune it on the same dataset
- Compare BERT's performance against your two models
- What difference do contextual embeddings make?

Deliverable

A single Jupyter notebook (.ipynb) that includes:

- All code, clearly commented
- Markdown cells explaining each decision
- All plots and evaluation metrics
- A short conclusion (10–15 lines) summarising what you learned

Evaluation Criteria

Criteria	What We Look For
Preprocessing	Correct steps applied with clear reasoning

Analysis	Meaningful observations from the data
Modelling	Both models built and trained correctly
Evaluation	Correct metrics used and interpreted properly
Explanation	Can you explain what you did and why?
Code Quality	Clean, readable, and well-commented code

 *Tip: The code matters less than your understanding. We are more interested in why you made each decision than whether your accuracy is high.*